

A light-weighted Viterbi Decoder implemented by FPGA

Zhenzhi Wu, Shujuan Hou, Hai Li

School of Information and Electronics,
Beijing Institute of Technology,
Beijing, China

e-mail: {wuzhenzhi, shujuanhou, haili}@bit.edu.cn

Abstract—In this paper, a light-weighted pipelined serial Viterbi Decoder is implemented for resource saving purpose. The traceback module of the decoder consumes fewer logical resources by employing a RAM-based Register Exchange architecture. All the metric and traceback bits are stored in the RAM to save logical resources. Synthesis results show that, the proposed architecture can save more than half of resource utilization than fabric IP core and has the minimum logic consumption than almost other schemes we can find with nearly no performance loss.

Keywords—Viterbi Decoder; Register Exchange; Resource efficient

I. INTRODUCTION

Viterbi decoding algorithm [1] has been widely adopted for convolutional code decoding. In some application, where the Viterbi decoder is implemented in FPGA and the chip cost is mainly concerned, the resource utilization should be carefully limited.

Some area-efficient implementation schemes are proposed in [2] [3] [4] which are based on parallel architecture. Such architecture can obtain a higher throughput, however, consumes relatively large amount of resources and unacceptable in our application scenario. In another aspect, some serial architecture schemes are proposed in [5] [6] [7], however, they use Trace Back method which doesn't make good use of the serial processing architecture, and the Survivor Memory Unit can't be highly optimized for resource saving purpose.

In our proposal, an extremely low resource utilization serial Viterbi Decoder is implemented in FPGA for low cost application. The register and logic resources are saved by RAM-based metric and traceback bit storage, and the FPGA chip cost can be lowered. Because serial architecture suffers from low throughput, 6-stage no-stall pipeline architecture is employed to maximize the throughput.

In a typical implementation scheme, the Viterbi Decoder is mainly composed of 3 parts. The Branch Metric Unit (BMU), the Add Compare Select (ACS) and the Survivor Memory Unit (SMU). The BMU is used for calculating code distance between received data and local convolutional code, where the resource is mainly consumed by Euclidean distance calculation and the metric storage. The SMU is used for extract the decoded bits after the traceback length, where the resource is consumed by survivor sequence (or path)

storage and decoding, and much more resources are consumed than the other two blocks. Thus, in our proposal, the SMU is highly optimized in order to obtain low resource consumption. In addition, resource efficient methods for BMU are also considered.

Our design is mainly based on the following criteria:

- (1) Multiplex BMU, ACS and SMU for each state and sequentially process each state;
- (2) Store all the data (traceback bits and metrics) in the memory, and a dual port RAM-based Register Exchange architecture is employed;
- (3) Simplify the metric calculation algorithm;
- (4) Scheduling a 6-stage no-stall pipeline to get the maximum throughput.

In the following paper, TOTALSTATE denotes the total states of one stage in the Viterbi decoding trellis diagram, which is equals to $2^{\text{constraint length}}$. SN represents the total bits needed to represent the TOTALSTATE, equals to the constraint length, and TN represents the traceback length which should be at least equals to 6 times of constraint length. CC(n,1,K) represents convolutional code which has constraint length K and code rate 1/n.

II. MODIFIED REGISTER EXCHANGE METHOD BASED ON LONG DATA WIDTH DUAL-PORT RAM

A. Modified Register Exchange Method

There are mainly two methods to extract the decoded sequence in survivor path: the Register Exchange (RE) method and Trace Back (TB) method [8]. In RE method, the register for each state records all the decoded sequences in constraint length. When decoder transitions to a new stage, the records should be copied from the old states to the new states. It doesn't need the traceback process compared to TB method, however, registers in all the status form a large register array which consumes large amount of resources. In TB scheme, the decoded sequence is decided by a tracing operation. It begins from the state with minimum path metric, follows the survivor path backward in time, and the unique path is identified, and a last-in-first-out (LIFO) buffer has to be used for reverse the decoded sequence. Although the register array is abandoned, the complicated traceback algorithm can also consume many resources.

In our proposal, we designed a modified Register Exchange architecture which stores the traceback bits in

RAM instead of register array, and no traceback operation is needed.

In the RAM-based RE method, all the decoded bits of one state in one stage are parallel stored and read by using a relatively long data width bus RAM (note as ‘Traceback RAM’). The width of RAM data bus is equal to the traceback length and the RAM depth is equal to the number of the states in one stage. The Traceback RAM inner architecture is depicted in Figure 1. It can output traceback bits of one state (previous state) according to the read address at single clock cycle with one clock delay. Because the registers are much more resource exhausted than RAMs, the resource utilization is reduced.

After ACS, the survival path is obtained and the index of the old state in the survival path is used as the index to address the Traceback RAM and all the decoded bits belonging to this state is fetched, the oldest bit at the head of the bit array is send to the output register, and the newly decided bit from ACS module is appended to the tail, in this way, the traceback operation is done with nearly no resource cost, and the new data bit array is formed and stored back to the RAM. The write address is the index of the current state.

B. Dual-Port RAM under pipeline with no data hazard

Because the state in current stage may be equal to the state in the previous stage, the write address of Traceback RAM equals to the read address and makes hazard, Odd-Even RAM architecture should be used to distinguish it. We propose an optimum method by using a single Dual-Port RAM which has two banks storing survival sequences of all states in adjacent stages. It has one write port and one read port, and the highest address bit is used to distinguish the current and previous stage. When changing from current stage to the next, the highest address should be changed, and the (previous stage) read RAM Bank changes into the (current stage) write RAM Bank, and the (previous stage) write RAM Bank changes into the (current stage) read RAM Bank. The Dual-Port RAM-based RE circuit is depicted in Figure 2.

Different from traditional RE method, where the register array can supply the traceback bits of all the states in one clock cycle, a Traceback RAM can supply traceback bits of only one state at each clock cycle. Luckily, it just suitable for the serial processing architecture, after carefully pipeline scheduling, it doesn’t need additional clocks and can supply equal throughput of BMU and ACS module. A stage counter is used to indicate the current stage index, and the state counter is accumulated with clock. After state counter finish traversing all the states in one stage, the stage counter is accumulated. The highest bit of stage counter is used to distinguish the Write and Read RAM Bank.

III. BMU AND ACS

A. Quantization and metric calculation

The state counter will traverse all the states of current stage, for each state, the corresponding 2 previous states are deduced, the old metric of ‘state a’ and ‘state b’ should be fetched and the ‘distance a’ and ‘distance b’ should be

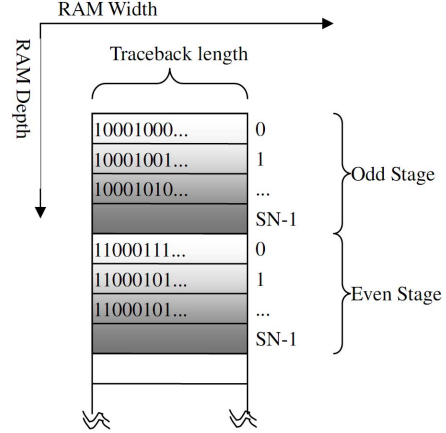


Figure 1. Traceback RAM architecture

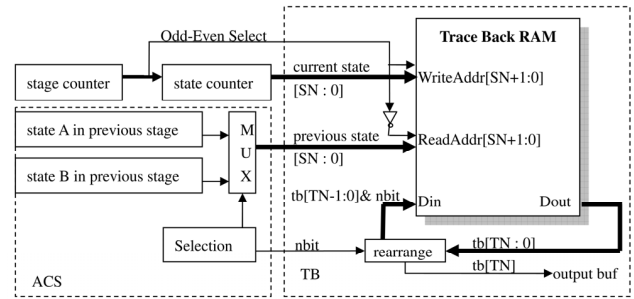


Figure 2. Traceback circuit

calculated respectively. The ‘distance a/b’ is the Euclidean distance between the local code word and received code word. We select 8-level quantization which suffers little performance loss but benefits for minimizing resource utilization. The Manhattan distance [4] instead of Euclidean distance is used for metric calculation because it is more resource efficient in hardware implementation and have an acceptable performance loss (about 0.2dB).

B. Two Dual-Port metric RAM under pipeline for two previous states

In ACS module, the current metric should be added to the metric of states in previous stage, the sum of the results will be compared and the survivor path is selected. The previous path metric can be stored in registers, however, in order to lower logic utilization further, the path metrics of all states are stored in ‘Metric RAM’. Unfortunately, different from Traceback RAM, which only needs to read value of **one** state in a clock cycle, Metric RAM must supply metric values of **two** old states belonged to different branch paths, which is required by the pipeline architecture. Different solutions are adopted based on different FPGA series. For Altera series, using 3 Port RAM, we can write one data and read two data in one clock cycle. For Xilinx series, which doesn’t support 3 Port RAM, the Metric RAM has to be copied into two RAMs, which are written with the same address and read with different address.

IV. PIPELINE SCHEDULING

For area-efficient purpose, only one execution unit is employed in BMU, ACS and SMU, and the fastest calculation can be guaranteed by no-stall pipeline, which means finishing all the operations of one state in one clock. The operation includes: The fetch and store of metric, the ACS operation, the read/write and renew of traceback bits, output the bit of the state that have the minim metric.

The pipelined Viterbi Decoder circuit is shown in Figure 4. Because the Metric RAM supply the path metric after one clock cycle, other signals has to be delayed by one clock, the same occasion occurs for the Traceback RAM. Thus we divide pipeline using register array <2> and <4>, the register array <1> is used for the state and stage update, and also for input data registering. Register array <5> is used for waiting the Output Buffer's bit, which is belonged to the minimum metric path. Although there is no logical error when register array <3> is not employed, it also gives a great benefit on dividing the critical path introduced by ACS operation, which starts from the Metric RAM data port and ends to the Metric RAM write port, the maximum frequency has raised about 30Mhz in Xilinx Spartan II series after employing it. As a result, totally 6 stages are introduced.

V. PERFORMANCE ANALYSIS AND SIMULATION

A. Throughput and Latency

The total throughput of this method can be calculated as following:

$$\text{Throughput(bits/s)} = \text{ClkFreq} / \text{TOTALSTATE} \quad (1)$$

where ClkFreq is the frequency of the clock.

It is accepted when the TOTALSTATE is not very large, for example $\text{TOTALSTATE} = 4$ to 64 when the constraint length is 3 to 7.

The latency can be calculated as following:

$$\text{Latency(cycles)} = \text{TOTALSTATE} \times \text{TracebackLength} + \text{InitTime} \quad (2)$$

Where the InitTime is as long as TOTALSTATE cycles which is used to clear the content of Metric RAM.

B. Bit Error Rate Performance under AWGN channel

We also build a bit-accurate model in Matlab and compared to Matlab function 'vitdec', and the BER performance is compared in Figure 3, the results shows there is nearly no performance difference between our design and Matlab function. In the simulation, we use 500000 BPSK symbols for Monte Carlo simulation, and the code rate gain has been counted in, which means $\text{SNR} = E_b/N_0 - 10\log_{10}(1/\text{Code Rate})$.

VI. SYNTHESIS RESULTS

This design is targeting on using as less resources as possible, which can be compared to other approaches. We firstly choose the Viterbi Decoder IP core provided by Altera [9] and Xilinx [10] for comparison. We select the configuration which has the minimum resource utilization in the datasheets, and using same parameter (constraint length of 5, 8-level quantization), and the same FPGA devices. The

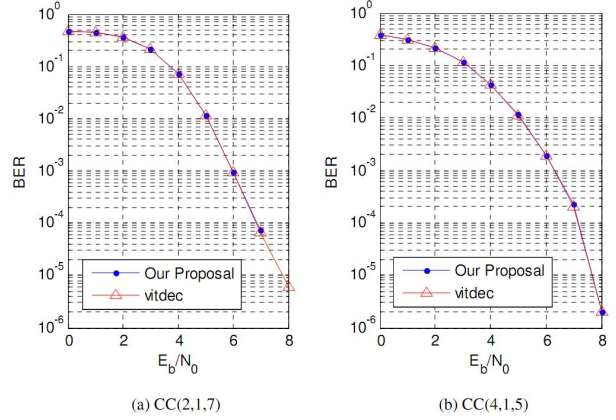


Figure 3. Performance comparison between Matlab 'vitdec' function and our proposal

TABLE I. XILINX DEVICE COMPARISON RESULT

Parameter	Logicore	Our Proposal
Area(slices)	261	116
Traceback Length	30	32
Block RAMs	2	5
Latency	129*11=1419	32*16+32=544
Max Freq	260 MHz	180 MHz
Mbps	260/11=23	180/16=11.25

TABLE II. ALTERA DEVICE COMPARISON RESULT

Parameter	Megacore	Our Proposal
Total LEs	-	225
Combinational LUTs	605	216
Logic Registers	387	118
Traceback Length	30	30
Total memory bits	M9K*5	1536
Max Freq	193	189.97 MHz
Mbps	19	189.97/16 = 11.87

Xilinx version is synthesized by ISE 7.1i compared to Logicore VD 5.0 using XC2V3000-6, and Altera version is synthesized by Quartus II 7.1 compared to Megacore IP Viterbi Compiler V9.1 using Device Cyclone III. The compare results are shown in TABLE I-II, our scheme consumes only 44% slices compared to Logicore, 36% LUT and 30% amount of registers compared to MegaCore IP. The throughput performance loss is relatively lower than manufacture IP due to the full serial processing architecture.

Some literatures have proposed Viterbi Decoder with different architectures. We collect most of them which have the resource utilization report, and comparisons to our design using the same or similar devices and parameters are presented. From TABLE III, we can find our proposal has the lowest resource consumption.

VII. CONCLUSION

We have proposed an extremely low resource consumption pipelined architecture for Viterbi Decoder, which is very suitable for the cost sensitive occasions. It has been shown that it consumes nearly no logical resource in extracting the decoded sequence, compared to traditional

TABLE III. COMPARISON TO OTHER SCHEMES IN THE LITERATURES

Scheme	Devices	Parameter	Resource Utilization	Our Proposal
Zhu[11]	XCV300	CC(2,1,7)	1137 Slices	122 Slices
Han[12]	XC2V2000	K=7 L=36	8090 Slices	122 Slices
Nandula[4]	Xilinx Virtex 2	CC(2,1,7) etc.	5261 slices	122 Slices
Keawsai[3]	xc2v1000-6	R= 1/2	1370 slices	122 Slices
Shaker[13]	2vp30fg676-6	CC(3,1,5)	376 slices	115 Slices*
Yang[14]	Stratix	CC(2,1,7)	2300 LEs	174 LEs
Wang[15]	Stratix	CC(2,1,7) L=48	522 LEs	174 LEs
Huang[16]	EP1C12Q240C8	CC(2,1,7)	400 LEs	174 LEs
Liu[5]	EP20KE200EFC 484-2X	CC(2,1,6)	2195 LEs	230 LEs*

*Note 1. Our proposal hasn't implemented CC(3,1,5) and CC(2,1,6), thus we use CC(4,1,5) and CC(2,1,7) instead which consume more resources for comparison.

Note 2. In the table, K represents the constraint length, L represents the traceback length, and R represents the code rate.

extracting the decoded sequence, compared to traditional Viterbi decoders both in Register Exchange and Trace Back architecture. Using pipelined architecture, it can output one data in TOTALSTATE cycles, and thus has a throughput of more than 10 MHz when using relatively short constraint length.

REFERENCES

- [1] Viterbi, A., *Error bounds for convolutional codes and an asymptotically optimum decoding algorithm*. Information Theory, IEEE Transactions on, 1967. **13**(2): p. 260-269.
- [2] El-Dib, D.A. and M.I. Elmasry, *Modified register-exchange Viterbi decoder for low-power wireless communications*. Circuits and Systems I: Regular Papers, IEEE Transactions on, 2004. **51**(2): p. 371-378.
- [3] Keawsai, C., K. Sripimanwat, and A. Lasakul, *Modified Register Exchange Method of Viterbi Decoder for CDMA2000 Mobile System* 2005.
- [4] Nandula, S., Y.S. Rao, and S.P. Embanath, *High speed area efficient configurable Viterbi Decoder for WiFi and WiMAX systems*. in *Intelligent and Advanced Systems, 2007. ICIAS 2007. International Conference on*. 2007.
- [5] LIU, Y., et al., *A FPGA Design of Serial Viterbi Decoder*. Chinese Journal of Electron Devices, 2007. **30**(5).
- [6] Petrov, M. and M. Glesner, *A state-serial Viterbi decoder architecture for digital radio on FPGA*. in *Field-Programmable Technology, 2005. Proceedings. 2005 IEEE International Conference on*. 2005.
- [7] Shenoy A. and J. Peter N. Johnson, *Serial Viterbi decoder*, C.S. Corporation, Editor. 1984: U.S.
- [8] Truong, T.K., et al., *A VLSI design for a trace-back Viterbi decoder*. Communications, IEEE Transactions on, 1992. **40**(3): p. 616-624.
- [9] Altera, *Viterbi Compiler User Guide v9.1*. 2009: Altera.
- [10] Xilinx, *Logicore Viterbi Decoder v5.0*. 2004.
- [11] Zhu, Y. and M. Benaissa, *Reconfigurable Viterbi decoding using a new ACS pipelining technique*. in *Application-Specific Systems, Architectures, and Processors, 2003. Proceedings. IEEE International Conference on*. 2003.
- [12] Jae-Sun, H., K. Tae-Jin, and L. Chanh. *High performance Viterbi decoder using modified register exchange methods*. in *Circuits and Systems, 2004. ISCAS '04. Proceedings of the 2004 International Symposium on*. 2004.
- [13] Shaker, S.W., S.H. Elramly, and K.A. Shehata, *FPGA implementation of a configurable viterbi decoder for software radio receiver*. in *AUTOTESTCON, 2009 IEEE*. 2009.
- [14] Hao, Y. and Y. Xiao-Lang, *Design and implementation of a high-speed and area-efficient viterbi decoder*. in *Solid-State and Integrated Circuit Technology, 2006. ICSICT '06. 8th International Conference on*. 2006.
- [15] Wang, D. and J. Qin, *A Implementation of a Non backtracking Viterbi Decoding Algorithm in FPGA* J RADIO ENGINEERING OF CHINA, 2007. **37**(4): p. 3.
- [16] Huang Hua-Zhu, Liu Rong-Ke, and Wang Run-Xin, *A FPGA implementation of serial architecture (2,1,7) convolutional code Viterbi Decoder*. Remote sensing and control, 2009. **30**(3) (in Chinese).

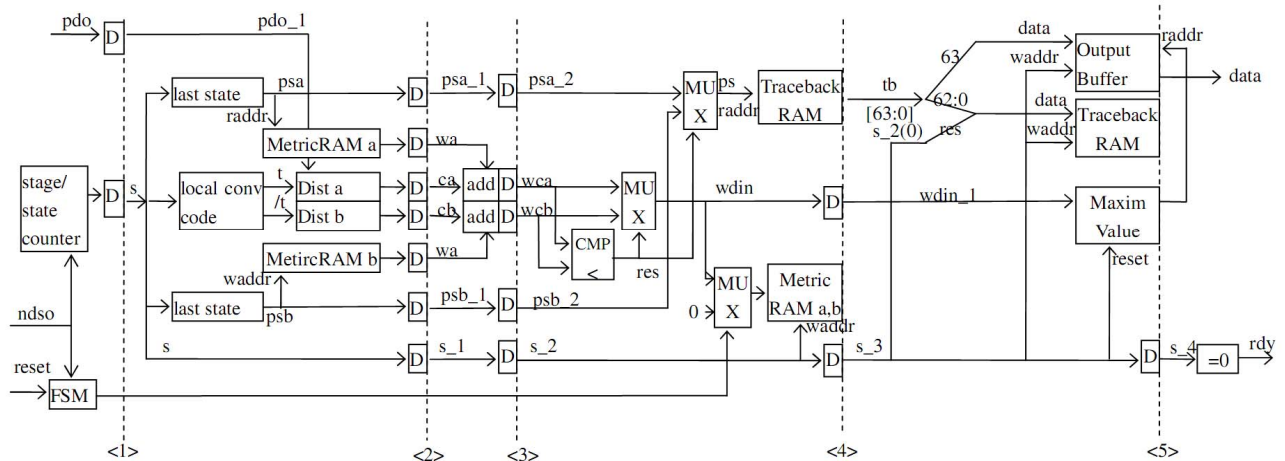


Figure 4. Viterbi Decoder pipeline architecture